

第7章 代码优化

7.1 概述

- 中间代码是一种通用的抽象，不考虑与具体机器相关的细节内容。
- 高级语言编写的程序可能本身并不是优化的。
- 某些高级语言的特征在映射到更低层面语言时，存在多种实现方式，具有不同的时间、空间开销。

- 优化必须保证语义上的等价
 - 在具有相同功能的条件下，使系统在某方面具有更好的指标。
- 优化的指标：
 - 目标代码指令条数
 - 目标代码运行效率
 - 目标代码占用内存的多少
 - 目标代码的功耗
 -

- 优化的时机
 - 中间代码生成时的优化
 - 平台无关的优化
 - 平台相关的优化(很多在代码生成阶段进行考虑)
- 本章主要关注代码优化的基本原理，不考虑具体机器的情况。

7.2 中间代码生成阶段的优化

- 大部分优化是在生成中间代码后进行的
- 在中间代码生成阶段也可进行部分优化
 - 布尔表达式的优化。
 - 控制流语句的少量优化。
 - 内置调用语句的优化，可根据已经知道的调用规律，减少不必要的开销。

7.3 代码优化的基本原则、思路 and 范围

- 代码优化的基本原则

- 等价原则：所谓等价原则，就是优化前后的代码具有完全相同的功能。
- 有效原则：优化后产生的代码达到预期的优化目标。
- 合算原则：以较小的代价获得尽可能高的优化效果。

优化的基本思路

- (1) 冗余代码的消除
- (2) 重复计算的合并与提取
- (3) 复杂计算的简单化

优化的范围

- 子程序内部的优化

- 第一，局部优化。该范围优化针对一个基本块内的中间代码进行优化。
- 第二，超局部优化和区域优化。该范围的优化考虑多个基本块之间的控制流关系，能够实现跨越多个基本块边界的优化。
- 第三，全局优化，也称过程内优化。是针对一个完整的子程序进行的优化。全局优化的主要技术是数据流分析技术。

7.4 基本块及其优化方法

- 所谓基本块，就是只包含线性无谓词代码的最长片段。
- 基本块是符合以下性质的代码序列：
- (1) 一个基本块有唯一的入口和出口，分别对应于该块的第一个和最末一个指令；
- (2) 基本块内部的所有指令都是顺序执行的，**不存在任何转跳**(除了出口语句可以指向其他基本块的入口)；

基本块划分算法

- 由于基本块中不包含指向内部的转跳语句，因此分支语句的转跳目标点只可能是一个基本块的开始；而分支语句本身是某个基本块的出口。
- 如果在两个入口语句之间既没有转跳语句作为出口，也没有其他入口语句，则第一个入口语句到第二个入口语句之间的所有语句属于一个基本块。

基本块划分算法

(1) 确定所有的基本块入口点

- 有三种情况：
- 第一，程序的第一个四元组是一个基本块的入口点；
- 第二，条件转移或无条件转移所指向的四元组或指令是一个基本块的入口点；
- 第三，紧跟在条件转移四元组之后的四元组是一个基本块的入口点。

(2)从每个入口四元组或指令出发，分别确定各基本块包含的所有四元组。具体判断方法如下：

- 第一，如果从一个入口到一个出口之间没有其他出口和入口四元组，则该入口语句到出口语句间的所有四元组属于一个基本块；
- 第二，如果一个入口到下一个入口四元组之间没有其他出口语句，则二者之间的所有四元组或指令(不含下一入口)属于一个基本块；
- 第三，如果从一个入口到停机四元组之间不存在其他入口和出口，则该入口到停机四元组之间的所有四元组属于同一个基本块。注意，停机四元组也可以是表示子程序结束的return等语句。

基本块划分的例子1

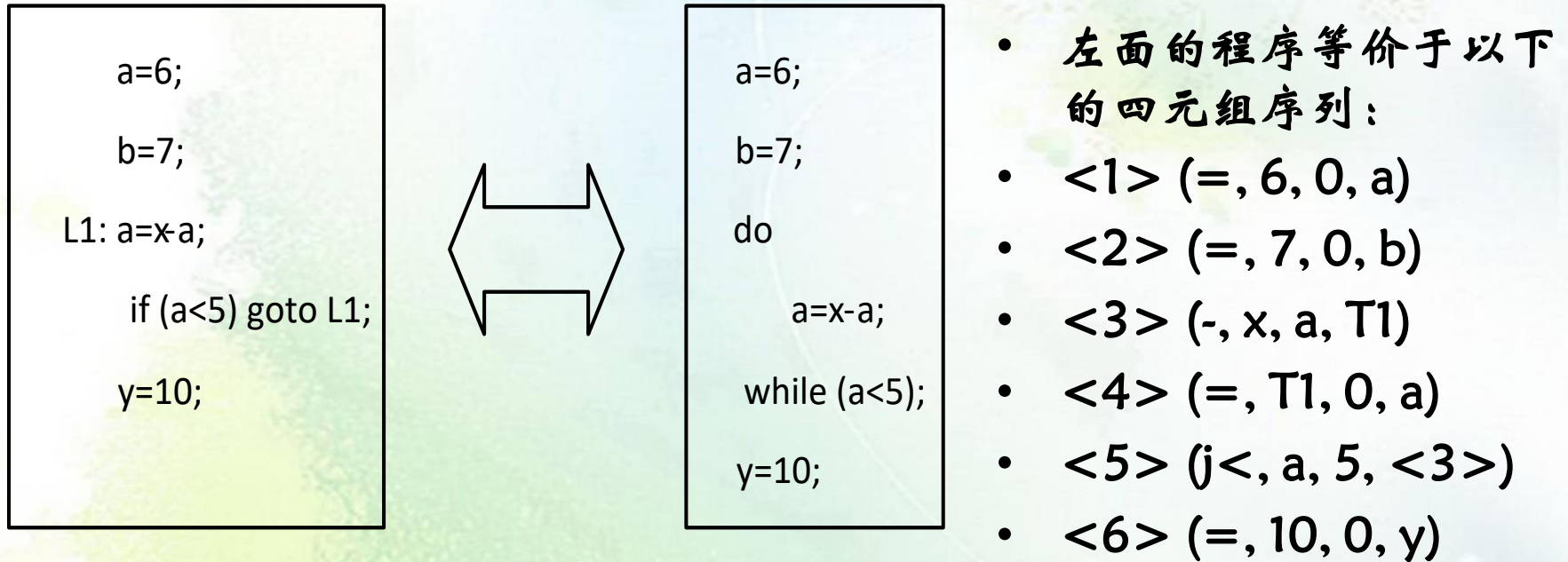


图 7.1 例 7.1 示意图

- **<1><2>是一个基本块。**

- 其判定依据是<1>是入口，<2>是<1>到下一入口<3>之间的四元组，且其间没有其他入口和出口四元组。

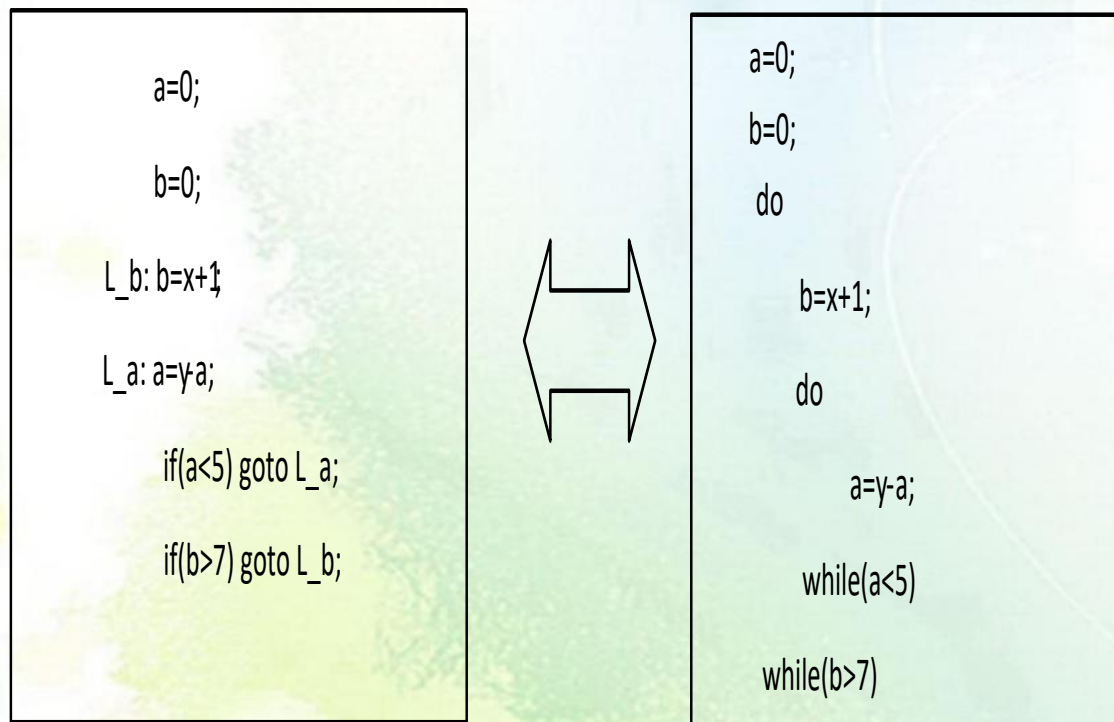
- **<3><4><5>是一个基本块。**

- 其判定依据是<3>是<5>的转跳目标，因此是一个入口点。<3>~<5>中，只有<5>是出口点，且没有停机语句、其他出口点或入口点。

- **<6>是一个基本块。**

- <6>是条件转移语句<5>的下一条语句，是一个入口点。如果没有其他后续语句，则<6>可以构成一个单独的基本块。

基本块划分的例子2



- 该程序写成等价的四元组序列如下:
- <1> (=, 0, 0, a)
- <2> (=, 0, 0, b)
- <3> (+, x, 1, T1)
- <4> (=, T1, 0, b)
- <5> (-, y, a, T2)
- <6> (=, T2, 0, a)
- <7> (j<, a, 5, <5>)
- <8> (j>, b, 7, <3>)

图7.2 例7.2 示意图

- 对该四元组序列划分基本块,可得到以下结果:
- $\langle 1 \rangle \langle 2 \rangle$ 属于同一个基本块;
- $\langle 3 \rangle \langle 4 \rangle$ 属于一个基本块(对应于 $b = x + 1$);
- $\langle 5 \rangle \langle 6 \rangle \langle 7 \rangle$ 属于一个基本块(对应于 $a = y - a$ 和 $\text{if}(a < 5) \text{ goto } L_a$);
- $\langle 8 \rangle$ 属于一个基本块(对应于 $\text{if}(b > 7) \text{ goto } L_b$)。

控制流图

- 基本块划分语句把一段代码序列划分为若干相互联系的基本块。如果把上面划分得到的基本块用有向边连接起来(有向边从一个基本块的结尾指向下一基本块的开头), 则得到程序的控制流程图。

控制流图

```
a=0;  
b=0;  
do  
  b=x+1;  
do  
  a=y-a;  
while(a<5)  
while(b>7)
```

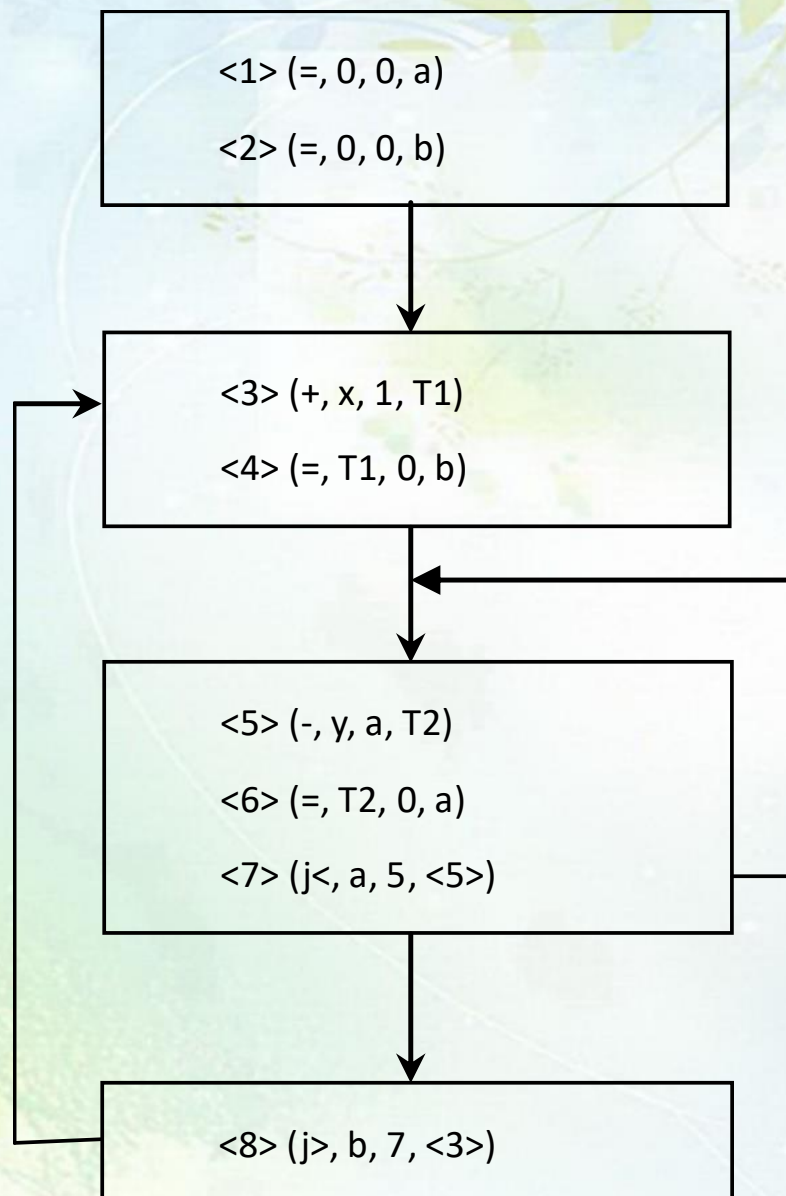


图 7.3 例 7.2 的基本块

- 为了解决表达式结合顺序所带来的优化难题，需要使用表达式重排序技术。很多编译器在设计时使用启发式技术来搜索更好的表达式排序方案，以便得到更有利于进行优化的表达式计算顺序。

7.4.1 基于值编号技术的基本块优化算法

值编号(Value Numbering)的思想

- 值编号的基本思想，是利用散列表的方式实现对公共表达式的识别。利用该技术，可以标识出两个等价的“冗余”表达式：即，如果两个表达式有相同的操作符，而且它们的操作数有相同的编号，则这两个表达式是等价的冗余表达式。
- 在使用值编号技术时，编译器需要维护一张散列表。其中，常量、变量直接用其文本信息进行散列，而对表达式的散列则针对其操作符和所有操作数的值编号共同生成一个散列值。根据这一定义，两个等价的冗余表达式必然具有相同的值编号，故该技术可用于方便地识别冗余表达式。

基本的值编号算法

- 对于基本块中的每个语句，执行以下计算(语句形式为 $\text{result} = \text{data1 op data2}$):
 - (1) 获取data1和data2的值编号;
 - (2) 根据操作符和data1、data2的值编号，产生唯一的散列值k;
 - (3) 若散列值k已经存在于散列表中，则说明该表达式是冗余的。此时，可以将该表达式的计算替换为对之前计算结果的引用(引用具有相同编号的变量)。另外，还需要将result标记为该散列值k对应的编号。
- 若k值不存在，则说明该表达式不是冗余的。此时将该散列值k插入散列表中，为其分配一个值编号。并用该值编号标记result。

值编号的例子

$$a=b+c$$

$$b=a-d$$

$$c=b+c$$

$$d=a-d$$

$$a^3=b^1+c^2$$

$$b^5=a^3-d^4$$

$$c^6=b^5+c^2$$

$$d^5=a^3-d^4$$

$$a=b+c$$

$$b=a-d$$

$$c=b+c$$

$$d=b$$

A、原始的中间表示

B、值编号的中间表示

C、结果中间表示

基本值编号的缺点

$a=x+y$

$b=x-y$

$a=17$

$c=x+y$

$a^3=x^1+y^2$

$b^4=x^1-y^2$

$a^5=17^5$

$c^3=x^1+y^2$

$a=x+y$

$b=x-y$

$a=17$

$c=x+y$

A、原始的中间表示

B、值编号的中间表示

C、结果中间表示

- 上面的程序中，第三句改变了变量a的值编号。这样，当处理第四句 $c=x+y$ 时，虽然可以知道 x^1+y^2 的编号是3，但由于a在第三句中已经获得了新的编号4，因此无法用 $c=a$ 进行替换。

对基本值编号算法的扩展

- 对于每个变量，除了记录其值编号外，再增加一个版本信息。
 - 编译器为变量的每次赋值都起一个新的名字(原名加一个版本号——例如， a_0 是对 a 的第一次赋值，而 a_1 则是对 a 的第二次赋值)。

$$a_0^3 = x_0^1 + y_0^2$$

$$b_0^4 = x_0^1 - y_0^2$$

$$a_1^5 = 17^5$$

$$c_0^3 = x_0^1 + y_0^2$$



$$a_0 = x_0 + y_0$$

$$b_0 = x_0 - y_0$$

$$a_1 = 17$$

$$c_0 = a_0$$

7.4.2 基本块中的其他优化

1. 常数合并和常数传播

2. 无用变量和无用代码的删除

- 常数合并和常数传播可以通过对前面介绍的值编号算法进行扩展来实现

(1) 常数合并

- 如果某个表达式所包含的全部操作数都是常数，则可以将一表达式用这些常数操作数的计算结果来代替(仍然是一个常数)。例如：
 - $x = 3 * 5 + 4$
 - 即可替换为 $x = 19$ 。

(2) 常数传播

— 所谓常数传播，就是当一个变量或表达式被替换为一个等价的常数后，则基本块中在该替换执行后的任何对该变量或表达式的引用都可替换为这一常数(前提是该变量在这次引用之前没有重新定值)。例如：

— $x = 3 * 5 + 4;$

— $y = x * z;$

— 常数传播的结果是：

— $x = 19;$

— $y = 19 * z;$

常数合并和常数传播的计算

- 对常数合并和常数传播的计算，可以通过对值编号算法进行改造来实现

$T1=3*5$

$x=T1+4$

$b=x-y$

$a=17$

$c=x+y$

A、原始的中间表示

$T1^3=3^1*5^2$

$x^5=T1^3+4^4$

$b^7=x^5-y^6$

$a^9=17^8$

$c^{10}=x^5+y^6$

B、值编号的中间表示

$T1=15$

$x=19$

$b=19-y$

$a=17$

$c=19+y$

C、结果中间表示

- 上图为基于值编号扩展的常数合并与常数传播的例子。
- 其中，在进行值编号时不难发现，3、5都是常量，而 $T1=3*5$ 是一个常量赋值，故可得到 $T1=15$ 。
- 同样的， $x=T1+4$ 语句不难看出， $T1$ 是一个常量，4是一个常量，故可以得到 $x=19$ (对 $T1$ 进行常量替换)。
- 类似的，在 $b=x-y$ 和 $c=x+5$ 中不难看出， x 是19，故得到 $b=19-y$ 和 $c=19+y$ 。

无用变量和无用代码的删除

- 无用变量和无用代码对于程序的计算结果不产生任何影响，是可以被删除而不影响程序语义的多余计算。
- 数据流分析技术验证变量和代码的无用性。只有被真正证明是无用的代码，才能够被删除而不影响程序的原始语义。

(1) 无用变量

— 需要利用数据流分析中的活跃变量分析技术

```
x=u*v;  
t=9+d;  
x=3+t;  
y=x+15;
```

- $x=u*v$ 中的 x 是无用变量。这意味着对 $x=u*v$ 的计算结果，将不会对后续代码的执行产生任何影响。
- 故 $x=u*v$ 也属于无用代码。

(2) 无用代码

- 除了由于无用变量引起的无用代码外，还有两种常见的无用代码。一种是由于没有转跳语句指向而引起的永远不会执行的死代码；另一种则是由于条件永远为假而永不执行的代码段。

```
x=5;  
goto L1;  
y=6;  
L1:u=7;
```

```
if(true) x=5;  
else x=10;
```

